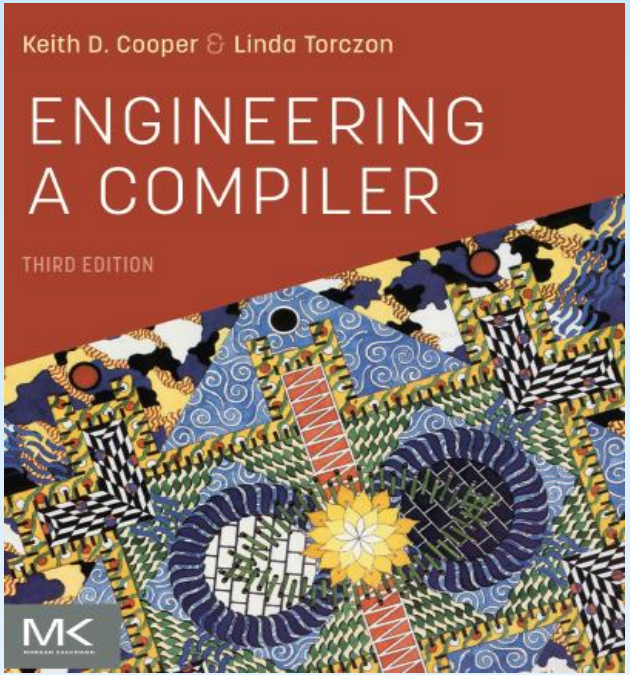
CSE212 Theory of Computation and Compilers

An Introduction !

Sara El-Metwally,
Associate Professor,
Computer Science Department

Course Details!



Course Code: CSE212

Course Name: Theory of Computation and Compilers.

Course TA: Eng. Besant Waheed.

Course Book: Engineering a Compiler, Third Edition.

Course Labs: Implementing some Compiler Phases.

Table of contents

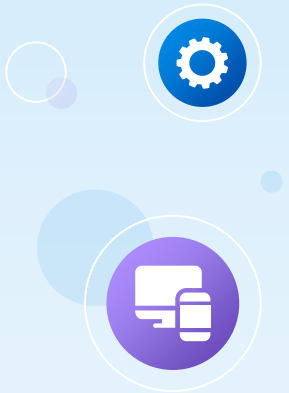
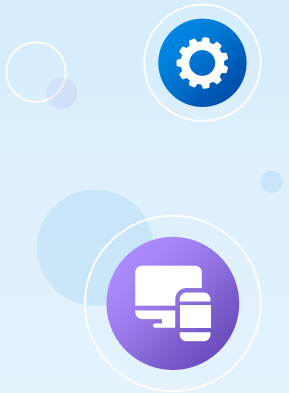
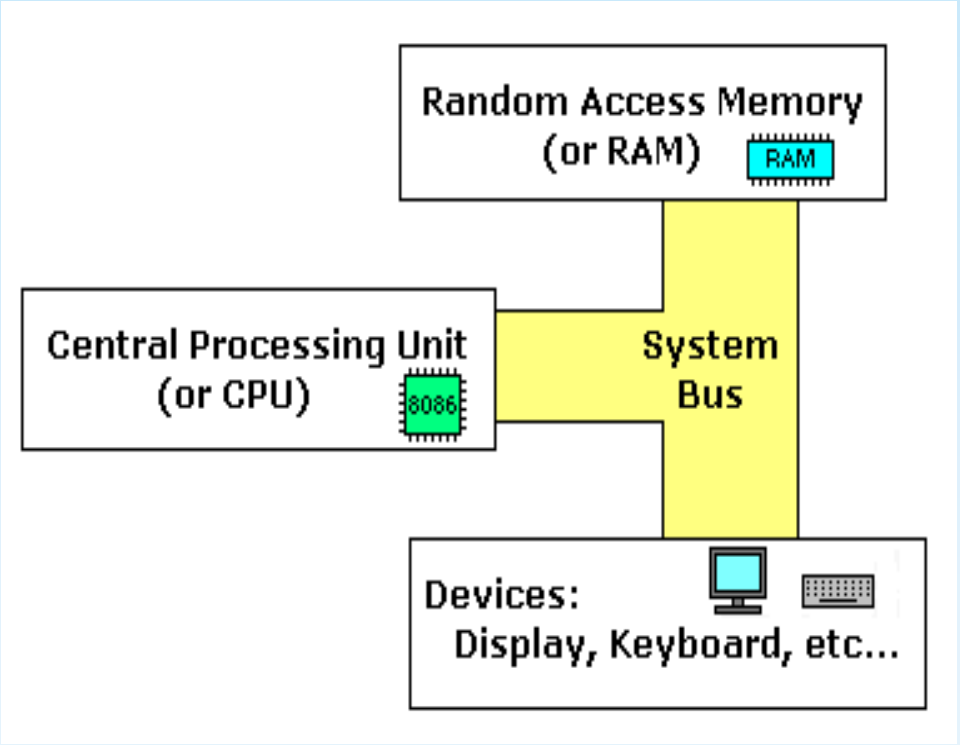


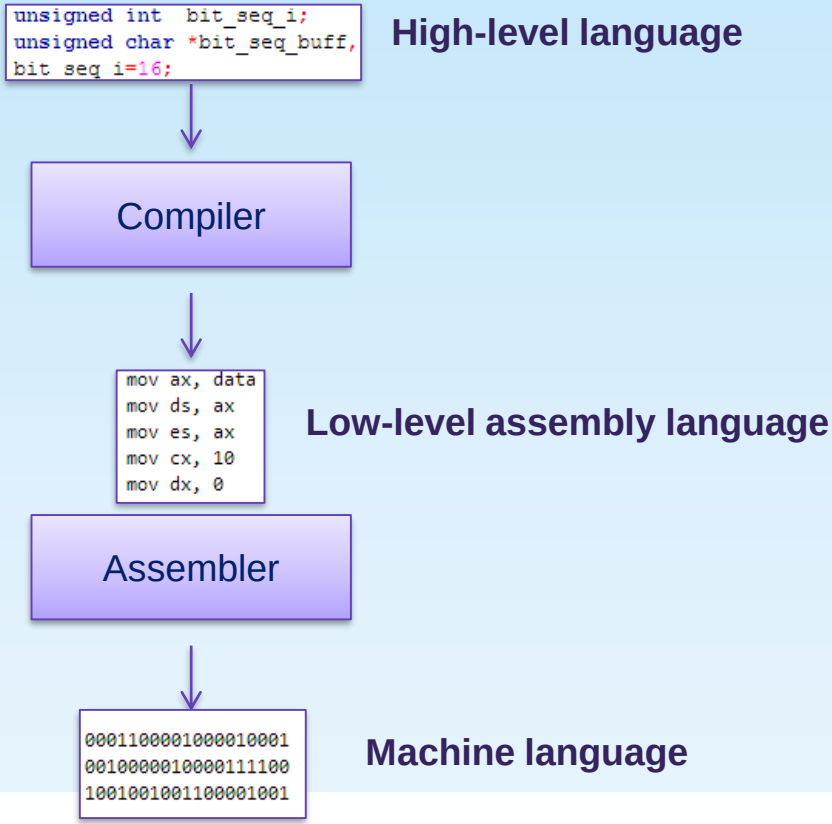
Table of contents



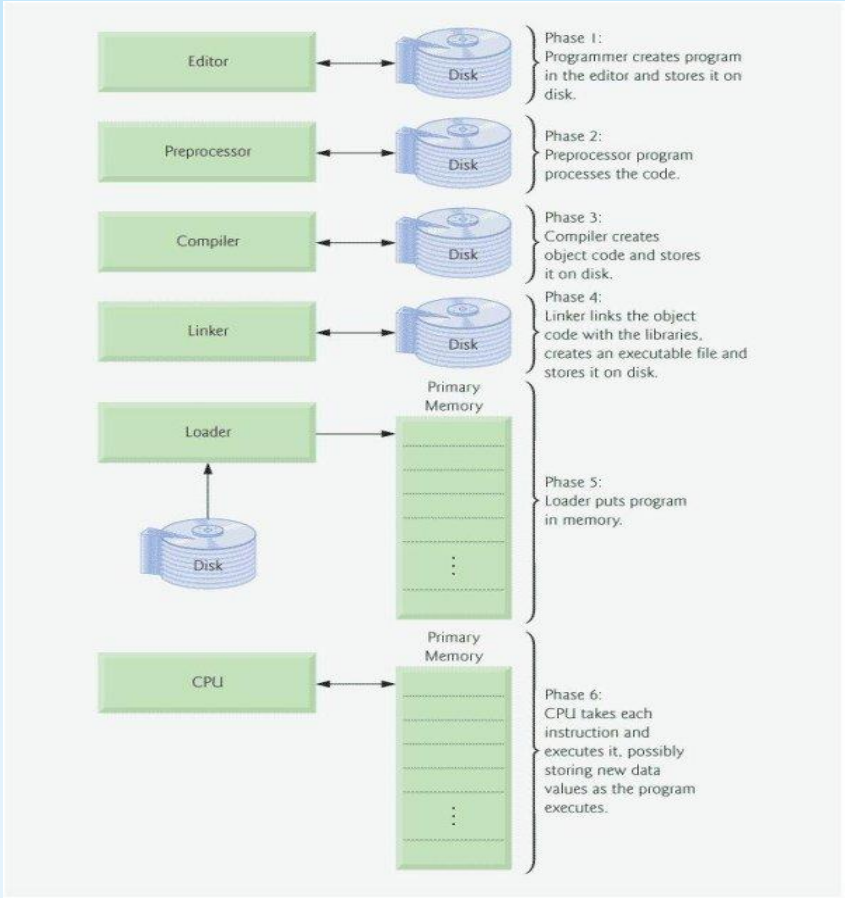
To Start:
You need to know your Computer ?



Levels of Programming



From Source to Executable ?



Notes

Term	Definition
Source Code	Source code is normally generated by humans (and thus hopefully able to be read and modified by humans) and is the human-readable representation from which all the other forms of code are created. A = B+C

Assembly Code	A low-level programming language code that requires software called an ***assembler*** to convert it into machine code.
	MOV R1, B
	MOV R2,C
	ADD R1, R2
	MOV A, R1

Notes

Term	Definition
Object Code	Object code is a portion of machine code that hasn't yet been linked into a complete program. It's the machine code for one particular library or module that will make up the completed product. It may also contain placeholders or offsets not found in the machine code of a completed program. The **linker** will use these placeholders and offsets to connect everything together.
Byte Code	Byte code (in Java, Python and etc.) is an intermediate code between source code and machine code that is executed by an interpreter such as JVM. e.g., Java class file. This makes byte code portable across multiple platforms (a combination of hardware and operating system) as long as a virtual machine is implemented on that platform.

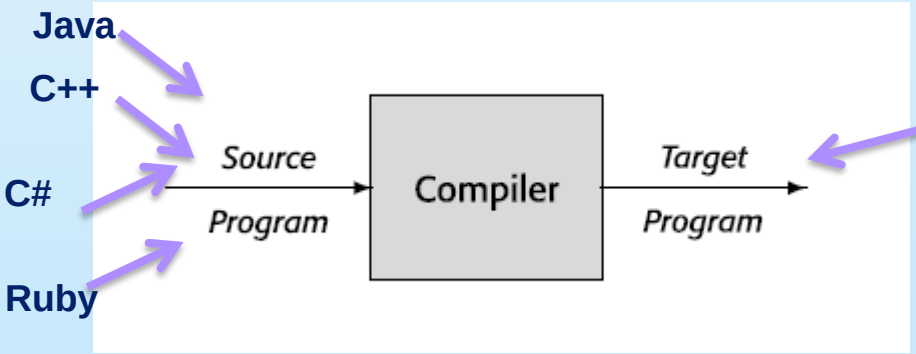
Notes

Term	Definition
Machine Code	Machine code is binary (1’s and 0’s) code that can be directly executable by the computer’s physical processor without further translation.
Executable file	The product of the linking process. They are machine code which can be directly executed by the CPU. e.g., a .exe file.
Library file	Some code is compiled into this form for different reasons such as re-usability and later used by executable files.

What is a compiler?

❑ A compiler is a computer program that translates a program written in one language into a program written in another language.

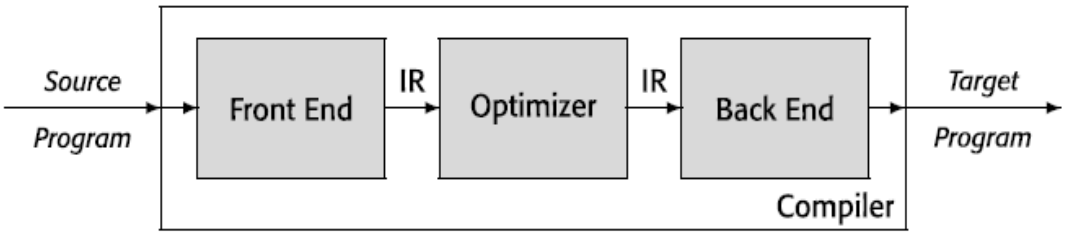
❑ A compiler is a computer program that translates other computer programs.



The assembly instruction set of some processors.

Compiler as a front– and back– ends

❑ The compiler has a front end to deal with the source language.
❑ It has a back end to deal with the target language.
❑ Connecting the front end and the back end, it has a formal structure for representing the program in an intermediate representation (IR), whose meaning is largely independent of either language.
❑ Most compilers include an optimizer that sits between the front end and the back end; the optimizer analyzes the IR and rewrites it into a form that is, under one or more metrics.



Source–to–source translators.

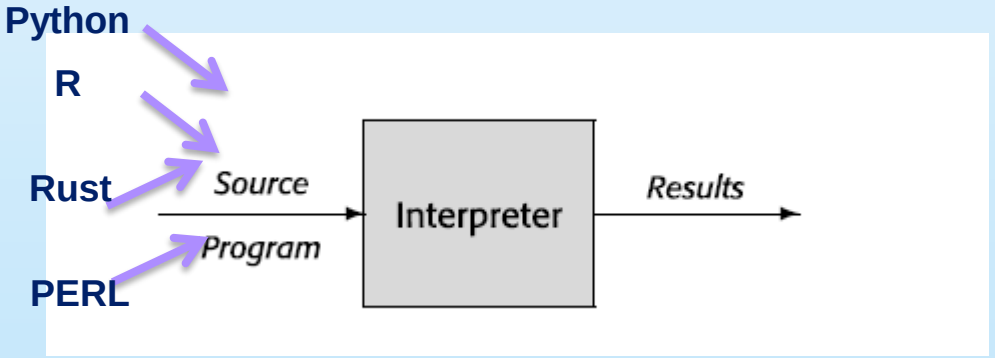
❑Some compilers produce a target program written in a human-oriented programming language rather than the assembly language and required further translation.

❑ Many research compilers produce C programs as their output. Because C compilers are available on most computers, this makes the target program executable on all those systems, at the cost of an extra compilation for the final target.

❑Compilers that target programming languages rather than the instruction set of a computer are often called *source-to-source translators.*

Compiler vs. Interepreter.

❑Interpreter creates machine code at runtime line by line [machine code on the fly]
❑ Compiler converts code at build time into a machine code.



Interpreter Vs Compiler

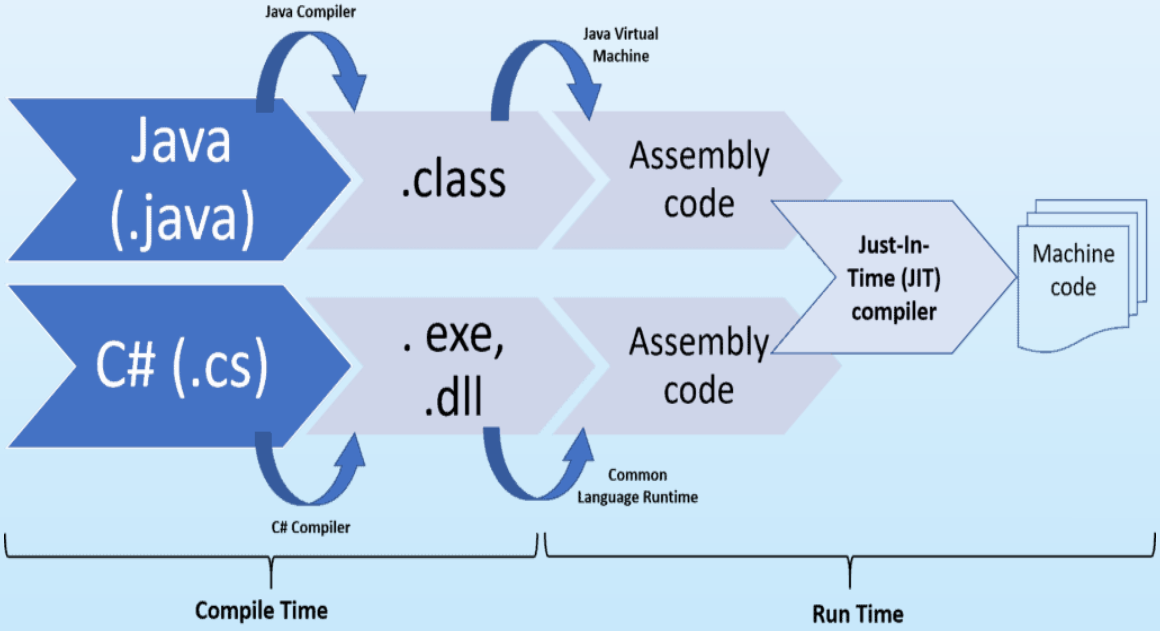
Interpreter	Compiler
Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
Interpreters usually take less amount of time to analyze the source code. However, the overall execution time is comparatively slower than compilers.	Compilers usually take a large amount of time to analyze the source code. However, the overall execution time is comparatively faster than interpreters.
No Object Code is generated, hence are memory efficient.	Generates Object Code which further requires linking, hence requires more memory.
Programming languages like JavaScript, Python, Ruby use interpreters.	Programming languages like C, C++, Java use compilers.

AOT vs. JIT

❑ In computer science, ahead-of-time compilation (AOT compilation) is the act of compiling an (often) higher-level programming language into an (often) lower-level language before execution of a program, usually at build-time, to reduce the amount of work needed to be performed at run time.

❑ In computing, just-in-time (JIT) compilation is compilation (of computer code) during execution of a program (at run time) rather than before execution.

AOT vs. JIT


Java Compiler
Java Virtual Machine
Java (.java)
.class
Assembly code
Just-In-Time (JIT) compiler
Machine code
C# (.cs)
. exe, .dll
Assembly code
Common Language Runtime
C# Compiler
Compile Time
Run Time

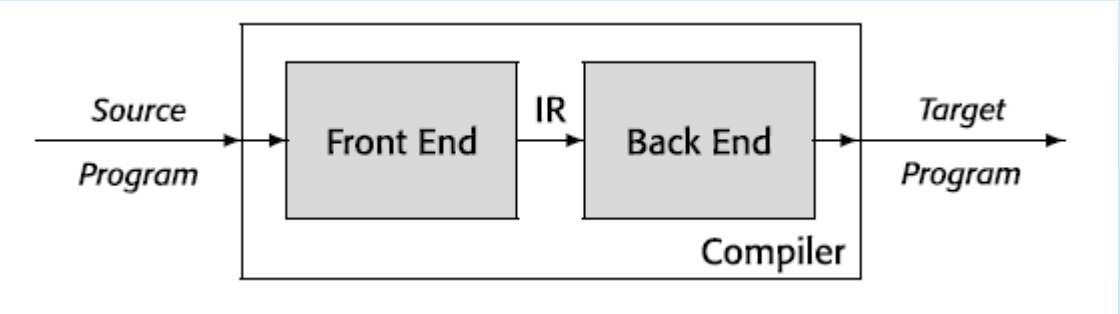
JIT

❑ JIT compilation can lead to complex translation schemes. For example, JAVA's execution model includes both compilation and interpretation. JAVA is compiled from source code into a more compact representation called JAVA bytecode by an AOT compiler.

❑ The application executes by running the bytecode on the JAVA Virtual Machine (JVM), an emulator, or interpreter, for JAVA bytecode. To improve performance, many implementations of the JVM include a JIT that compiles and optimizes heavily used bytecode sequences.

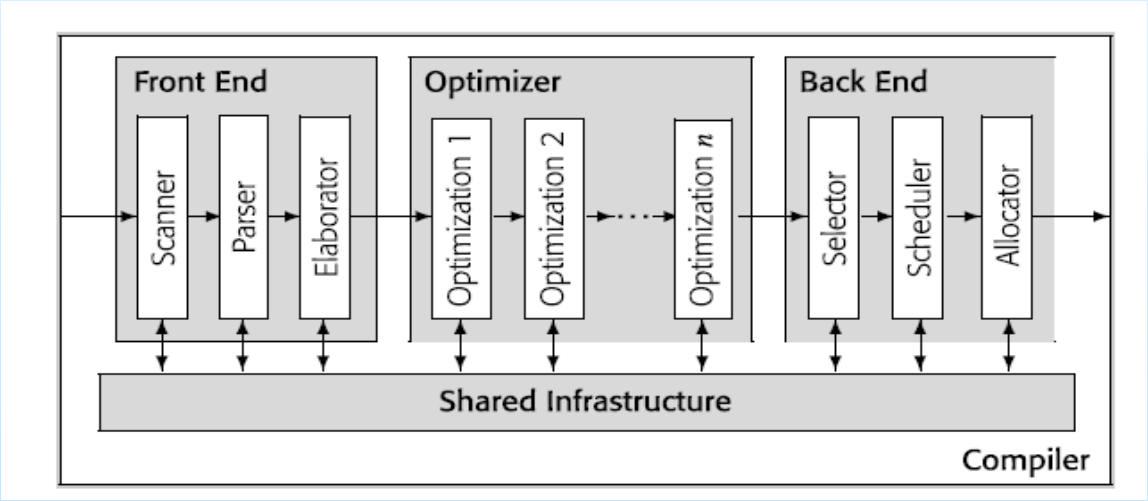
Front and Back – ends of Compilers

❑ The front end focuses on understanding the source-language program.
❑ The back end focuses on mapping programs to the target machine.
❑ A compiler uses some set of data structures to represent the code that it processes. That form is called an intermediate representation, or IR.
❑ The task of changing the compiler to generate code for a new processor is often called retargeting the compiler.



Dive into Compiler Phases.

❑ The middle section of a compiler, called an optimizer, analyzes and transforms the IR in an attempt to improve it.



Dive into Compiler Phases.

❑ The optimizer is an IR-to-IR transformer, or a series of them, that tries to improve the IR program in some way.
❑ The optimizer can make one or more passes over the IR, analyze the IR, and rewrite the IR.
❑The optimizer may rewrite the IR in a way that is likely to produce a faster target program from the back end or a smaller target program from the back end. It may have other objectives, such as a program that produces fewer page faults or uses less energy.

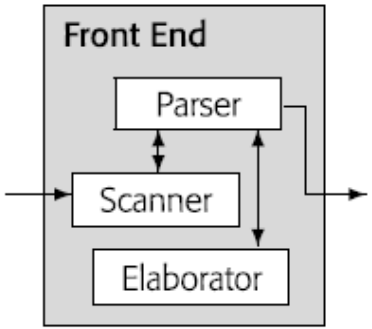
❑ Most compiler systems support separate compilation. The programmer can compile distinct files of code in independent steps, often at different times. Each of these compilations produces a file of object code. Multiple object code files can be linked together to form a single executable.

Front–end of Compiler

❑ **a ← a × 2 × b × c × d**

❑ Before the compiler can translate an expression into executable target machine code, it must understand both its form, or syntax, and its meaning, or semantics.

❑ The scanner converts the stream of characters from the input code into a stream of words. It recognizes valid words by their spellings; for example, in most programming languages, **1g2h3i** is neither a valid identifier nor a valid number.

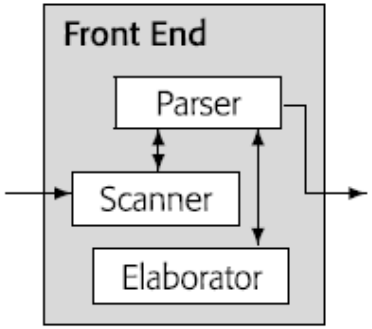


Front–end of Compiler

❑ The parser fits the words from the scanner to a rule-based model of the input language's syntax, called a grammar.

❑ *Sentence* → *Subject* verb *Object* endmark

❑ It calls the scanner incrementally as it needs additional words. As the parser matches words to rules in the grammar, it may call on the elaborator to perform additional computation on language the input program. Examples of such computation include building an IR, checking type consistency.



Front–end of Compiler

❑ Scanner: the compiler pass that converts a string of characters into a stream of classified words.

❑ Consider a sentence like “Compilers are engineered systems.”

❑ A scanner might convert the example sentence into the following stream of classified words: **(noun, “Compilers”), (verb, “are”), (adjective, “engineered”), (noun, “systems”), (endmark, “.”)**

❑ In the next step, the compiler tries to match the stream of words to the rules of the grammar.

Front–end of Compiler

❑ The following derivation for the sentence "Compilers are engineered systems."

1	*Sentence*	→	*Subject* verb *Object* endmark
2	*Subject*	→	noun
3	*Subject*	→	*Modifier* noun
4	*Object*	→	noun
5	*Object*	→	*Modifier* noun
6	*Modifier*	→	adjective

Rule	Prototype Sentence
–	*Sentence*
1	*Subject* verb *Object* endmark
2	noun verb *Object* endmark
5	noun verb *Modifier* noun endmark
6	noun verb adjective noun endmark

Front–end of Compiler

- The process of automatically finding derivations is called *parsing*.

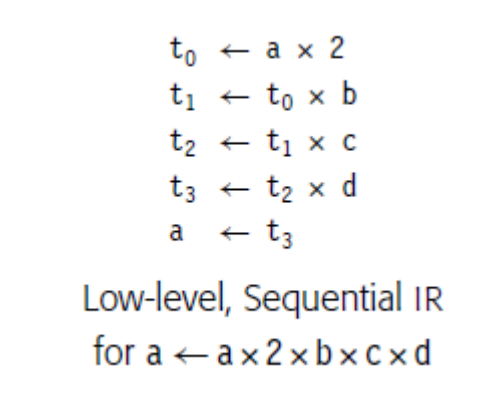
- A grammatically correct sentence can be meaningless. For example, the sentence "Rocks are green vegetables." has the same parts of speech in the same order as "Compilers are engineered objects." but has no rational meaning.

- Semantic elaborations expand the "meaning" of the program from simple syntax to an operational definition.

- Other errors can be detected in a semantic phase such as type mismatches, an array reference with the wrong number of dimensions or a procedure call with too many parameters.

IR of Compiler

❑Compilers use a variety of kinds of IR, depending on the source language, the target language, and the specific goals of the compiler. Some IRs represent the program as a graph. Other IRs resemble a sequential assembly code program.



❑ Efficiency can have many meanings. Classic code optimization tries to reduce the application's elapsed running time. In other circumstances, the optimizer might try to reduce the size of the compiled code, or the energy that the processor consumes as the code runs.

Optimization Phase of Compiler

- The elapsed time of a program refers to the amount of time that it would require to run standalone on a system.

- A value that does not change between iterations of a loop is said to be *loop-invariant.*

```
b ← ···
c ← ···
a ← 1
for i = 1 to n
    read d
    a ← a × 2 × b × c × d
    end
```

(a) Original Code in Context

```
b ← ···
c ← ···
a ← 1
t ← 2 × b × c
for i = 1 to n
    read d
    a ← a × d × t
    end
```

(b) Improved Code

The Compiler Back-End

- It must convert the IR operations into equivalent operations in the target processor's ISA, a process called *instruction selection.*

- It must select an execution order for the operations, a process called *instruction scheduling.*

- It must decide which values should reside in registers at each point in the code, a process called *register allocation*

Compiler Phases Recap

Phase	Output	Sample
Programmer (source code producer)	Source string	A=B+C;
Scanner (performs lexical analysis)	Token string	‘A’ , ‘=’ , ‘B’ , ‘+’ , ‘C’ , ‘;’ And symbol table with names
Parser (performs syntax analysis based on the grammar of the programming language)	Parse tree or abstract syntax tree	; | = / \ A + / \ B C
Semantic analyzer (type checking, etc)	Annotated parse tree or abstract syntax tree	
Intermediate code generator	Three-address code, quads, or RTL	int2fp B t1 + t1 C t2 := t2 A
Optimizer	Three-address code, quads, or RTL	int2fp B t1 + t1 #2.3 A

Code generator	Assembly code	MOVF #2.3,r1
		ADDF2 r1,r2
		MOVF r2,A

Peephole optimizer	Assembly code	ADDF2 #2.3,r2 MOVF r2,A

Thanks!